

第7章: 書籍管理アプリ — 一覧表示と登録機能の実装

いよいよアプリの本体を作ります！この章では、CRUD（Create=作成、Read=読み取り、Update=更新、Delete=削除）のうち、R（Read：一覧表示）とC（Create：新規登録）を実装します。

この章で作るもの

この章を終えると、以下の2つの画面が動くようになります。

画面	できること	CRUDのどれ？
書籍一覧ページ（トップページ）	Supabaseに保存された書籍をカード形式で表示する	R（Read：読み取り）
書籍登録ページ	フォームに入力して新しい書籍を登録する	C（Create：作成）

ユーザー体験の流れ

ユーザーの視点で、完成後の操作の流れを確認しましょう。

1. アプリを開く → **書籍一覧ページ**が表示される（登録済みの書籍がカード形式で並ぶ）
2. 「+ 新規登録」ボタンを押す → **登録フォーム**に移動する
3. タイトル・著者・評価などを入力して「登録する」を押す → データがSupabaseに保存される
4. 自動的に一覧ページに戻る → 今登録した書籍が一覧に追加されている

作成するコンポーネント（部品）一覧

コンポーネント名	役割	なぜ分けるのか
StatusBadge	「読書中」「読了」などのステータスを色付きバッジで表示	複数の画面で使い回すため
RatingStars	1～5の評価を星マーク（★）で表示	一覧・詳細の両方で使うため
BookCard	1冊分の書籍情報をカード型で表示	一覧画面でN冊分繰り返し表示するため
BookList	BookCardを並べて表示するコンテナ（入れ物）	レイアウトの責任を分離するため
BookForm	書籍の登録/編集に使うフォーム	登録と編集で同じフォームを使い回すため
LoadingSpinner	データ読み込み中の表示	ユーザーに「読み込み中です」と伝えるため

なぜコンポーネントを分けるの？ 1つの巨大なファイルに全てを書くこともできますが、部品に分けることで「見通しが良くなる」「バグを見つけやすくなる」「同じ部品を使い回せる」というメリットがあります。これはコンポーネント設計と呼ばれ、React開発の重要な考え方です。

目次

- 0. 前提知識: 非同期処理 (async / await) の超基礎
 - 1. 一覧表示機能の実装
 - 2. ローディング状態
 - 3. エラーハンドリング
 - 4. 登録機能の実装
 - 5. コード解説
 - 6. 動作確認手順
 - 7. トラブルシューティング

0. 前提知識: 非同期処理 (async / await) の超基礎

この章のコードでは `async` (エイシンク) と `await` (アウエイト) が頻出します。これは「時間がかかる処理を待つ」ための仕組みです。Supabaseに「データを取ってきて」とお願いする処理は、結果が返ってくるまで数十〜数百ミリ秒かかります。その「待ち時間」を扱うのが `async/await` です。

0.1 同期と非同期の違い

```
// 同期処理 (synchronous) : 上から下へ順番に実行され、即座に結果が返る
const sum = 1 + 2;
console.log(sum); // 3

// 非同期処理 (asynchronous) : 結果が返るのに時間がかかる
const data = supabase.from("books").select("*"); // ✗ これだけだと「Promise」というオブジェクトが返る
```

0.2 Promise って何？

非同期処理の「結果が返ってくる予定の入れ物」のことを **Promise (プロミス、約束)** と言います。

今すぐ返せる値: — 1 + 2 — 即 3 が返る
非同期の値: — DB問い合わせ — ⌚ Promise が返る — 数百ms後に中身が確定

0.3 async / await で「待つ」

`await` を関数呼び出しの前に書くと、「Promiseの中身が確定するまでこの行で待ってね」という意味になります。

`await` を使う関数には `async` を付ける必要があります。

```
// ❌ await なし: data には Promise オブジェクトが入る
const fetchBooks = () => {
  const data = supabase.from("books").select("*");
  console.log(data); // Promise { <pending> } と表示される
};

// ✅ async/await あり: data に実際のデータが入る
const fetchBooks = async () => {
  const { data } = await supabase.from("books").select("*");
  console.log(data); // [{ id: 1, title: "...", ... }, ...] と表示される
};
```

▼ 概念図:

```
async function fetchBooks() {
  const { data } = await supabase.from("books").select("*");
  //           └─ ここで「結果が来るまで待つ」
  console.log(data); // ← 結果が来てからこの行が実行される
}
```

0.4 try / catch でエラーを捕まえる

非同期処理は「サーバーが落ちている」「ネットワーク切断」などで失敗することがあります。`try / catch` で失敗を捕まえます。

```
const fetchBooks = async () => {
  try {
    const { data, error } = await supabase.from("books").select("*");
    if (error) throw error; // Supabaseが返したエラーは自分でthrow
    console.log("成功:", data);
  } catch (err) {
    console.error("失敗:", err); // ここに飛んでくる
  }
};
```

▼ 出力イメージ:

成功時:

```
成功: [ { id: 1, title: "...", ... }, { id: 2, ... } ]
```

失敗時（テーブル名を間違えた場合など）:

```
失敗: { code: '42P01', message: 'relation "bookz" does not exist' }
```

本書での使い方: Supabaseの `.from()` などすべて非同期なので、`async` 関数の中で `await` を付けて呼びます。エラーは `try/catch` で捕まえ、画面に「エラーが発生しました」と表示します。

1. 一覧表示機能の実装

一覧表示は、データベースに保存された書籍をカード形式で並べて表示する機能です。以下のコンポーネント構成で実装します。

```
app/page.tsx (トップページ / Server Component)
├── BookList (書籍一覧)
│   └── BookCard (書籍カード) × N冊分
│       ├── StatusBadge (ステータスバッジ)
│       └── RatingStars (星評価)
```

小さなコンポーネントから順に作っていきましょう。

1a. StatusBadge コンポーネント

ファイル: `components/StatusBadge.tsx`

書籍のステータス（読書中・読了・読みたい）をバッジ形式で表示するコンポーネントです。ステータスの値に応じて背景色とテキストが変わります。

```
// =====
// ファイルパス: src/components/StatusBadge.tsx
// 役割      : 書籍のステータス（読書中／読了／読みたい）を、
//             色付きの小さなバッジとして画面に表示するための部品（コンポーネント）
// 親コンポーネント: BookCard が使う
// =====

/**
 * StatusBadge - 書籍の読書ステータスをバッジ形式で表示するコンポーネント
 *
 * 表示されるバッジの種類:
 * - reading      : 「読書中」 （青色）
 * - completed    : 「読了」    （緑色）
 * - want_to_read : 「読みたい」（黄色）
 *
 * 受け取るデータ:
 * - status: 上記3種類のうちどれか1つ（文字列）
 *
 * 出力（画面に出るもの）:
 * - 角丸の小さなラベル（spanタグ）。文字色と背景色がステータスごとに違う。
 */

// -----
// (1) 「ステータスの型」を定義する
// -----
// TypeScript の「ユニオン型 (|)」を使って
// 「BookStatus は文字列だけど、3つのうちのどれか」というルールを作る。
//
// こう書くことで、誤って "reading2" など typo のある文字列を渡したときに
// VS Code がコードを書いている最中に赤線で警告してくれる。
// 「export」を付けているので、ほかのファイル（BookCardなど）からも import して使える。
export type BookStatus = "reading" | "completed" | "want_to_read";

// -----
// (2) このコンポーネントが「親から受け取るプロパティ (Props)」の形を定義
// -----
// Reactでは、親コンポーネントから子コンポーネントに渡すデータを「Props」と呼ぶ。
// ここでは「status」というキーで BookStatus 型の値を1つ受け取る、と宣言している。
//
// 例: <StatusBadge status="reading" /> ← こう呼び出される。
type StatusBadgeProps = {
  status: BookStatus;
};

// -----
// (3) ステータスごとの表示設定をオブジェクトでまとめる
// -----
// もし if文 や switch文 で「reading の場合は青、completed の場合は緑...」と
// 書くと、新しいステータスを追加するたびに分岐を増やす必要があり保守が大変。
```

```

//
// そこで「ステータス → { 表示文字, 背景色, 文字色 }」という辞書（オブジェクト）を
// 作っておくと、コンポーネント本体はこの辞書を引くだけで済む。
//
// `Record<BookStatus, ...>` は「キーは BookStatus のどれか、値は { ... }」という
// TypeScriptの記法で、辞書全体の型を保証する。
//
// `bg-blue-100` などは Tailwind CSS のクラス名。
//   bg-          = background-color
//   blue-100     = 100段階で薄めの青（数字が大きいほど濃い）
//   text-blue-800 = テキストカラーを濃い青に
const statusConfig: Record<
  BookStatus,
  { label: string; bgColor: string; textColor: string }
> = {
  reading: {
    label: "読書中",           // バッジに表示する日本語
    bgColor: "bg-blue-100",    // 背景: 薄い青
    textColor: "text-blue-800", // 文字: 濃い青
  },
  completed: {
    label: "読了",
    bgColor: "bg-green-100",    // 背景: 薄い緑（完了感のある色）
    textColor: "text-green-800",
  },
  want_to_read: {
    label: "読みたい",
    bgColor: "bg-yellow-100",   // 背景: 薄い黄
    textColor: "text-yellow-800",
  },
};

// -----
// (4) コンポーネント本体
// -----
// `export default` で「このファイルの主役はこの関数だよ」と宣言。
// 関数の引数を `{ status }` と書いているのは「分割代入」と呼ばれる JS の文法。
//   - 引数として渡されたオブジェクト (StatusBadgeProps型) から
//     `status` という名前のプロパティだけを取り出す。
//   - 同等の書き方:
//       function StatusBadge(props: StatusBadgeProps) {
//         const status = props.status;
//         ...
//       }
export default function StatusBadge({ status }: StatusBadgeProps) {
  // (4-1) 渡された status をキーにして、設定オブジェクトを引く。
  //
  // ?? は「Nullish Coalescing 演算子」と呼ばれ、
  // 左辺が null か undefined の場合に右辺を使うという意味。
  // 万が一 statusConfig に存在しないキー（不正な値）が来た場合のための保険。

```

```

const config = statusConfig[status] ?? {
  label: "不明",
  bgColor: "bg-gray-100",
  textColor: "text-gray-800",
};

// (4-2) JSX を返す。これが画面に描かれるHTML（厳密にはReactが生成するDOM）。
//
// <span> はインライン要素（行の中で使う小さな箱）。
// className に Tailwind のクラスを並べることで、CSSを別ファイルに書かずに
// スタイルを当てられる。
//
// ${...} の部分はテンプレートリテラル。
// configの値（変数）を文字列の中に埋め込んでいる。
return (
  <span
    className={`
      inline-flex items-center /* 中身（テキスト）を上下中央に揃える */
      px-2.5 py-0.5 /* 内側の余白（padding x方向 / y方向）*/
      rounded-full /* 角を完全に丸めてカプセル型に */
      text-xs font-medium /* 文字サイズを小さく、太さは中ぐらい */
      ${config.bgColor} /* 背景色（ステータスにより変わる） */
      ${config.textColor} /* 文字色（ステータスにより変わる） */
    `}
    >
    {/* {config.label} はJSXで「JS式を埋め込む」記法。
      ここでは "読書中" などの文字列が出力される */}
    {config.label}
  </span>
);
}

// =====
// この StatusBadge.tsx を <StatusBadge status="reading" /> のように書くと:
//
// 出力されるHTML（概念）:
// <span class="inline-flex ... bg-blue-100 text-blue-800">読書中</span>
//
// 画面の見た目:
//
// | 読書中 | ← 青背景・青文字の角丸ラベル
//
// =====

```

▼ どこから呼び出されるの？

このコンポーネントは、後で出てくる `BookCard.tsx` の中から次のように使われます。

```
import StatusBadge from "@/components/StatusBadge";

// ...
<StatusBadge status={book.status} />
// 例: book.status が "reading" のとき、青いバッジが描画される
```

画面にはこのように表示されます:

StatusBadge は小さな角丸のバッジとして表示されます。

- 「読書中」の場合: 薄い青色の背景に、濃い青色のテキストで「読書中」と表示されます。見た目は **[読書中]** のような角丸の小さなラベルです。
- 「読了」の場合: 薄い緑色の背景に、濃い緑色のテキストで「読了」と表示されます。完了を連想させる緑色です。
- 「読みたい」の場合: 薄い黄色の背景に、濃い黄色（オレンジ寄り）のテキストで「読みたい」と表示されます。「まだ読んでいない＝これから」を連想させる色です。

いずれも **text-xs**（12px 程度）の小さめのフォントサイズで、**rounded-full** により完全な角丸（カプセル型）になります。カードの中に配置すると、ステータスがひと目で分かるアクセントになります。

1b. RatingStars コンポーネント

ファイル: **components/RatingStars.tsx**

書籍の評価（1～5）を星マーク（★☆）で視覚的に表示するコンポーネントです。


```
// components/RatingStars.tsx

/**
 * RatingStars - 1~5の評価を星マークで表示するコンポーネント
 *
 * 例: rating=3 の場合 → ★★★☆☆
 *
 * rating が null の場合は「評価なし」と表示する
 */

type RatingStarsProps = {
  rating: number | null;
};

export default function RatingStars({ rating }: RatingStarsProps) {
  // 評価がない場合はテキストで表示する
  if (rating === null || rating === undefined) {
    return (
      <span className="text-sm text-gray-400">
        評価なし
      </span>
    );
  }

  // 1~5 の範囲に収める (データベースの制約と合わせる)
  const clampedRating = Math.min(5, Math.max(1, rating));

  return (
    <div className="flex items-center gap-0.5" aria-label={`評価: ${clampedRating}点`} >
      { /* 星を5つ並べる。rating以下のインデックスは塗りつぶし、それ以外は空の星 */ }
      {[1, 2, 3, 4, 5].map((star) => (
        <span
          key={star}
          className={`text-lg ${
            star <= clampedRating
              ? "text-yellow-400" // 塗りつぶしの星 (黄色)
              : "text-gray-300" // 空の星 (グレー)
          }`}
        >
          ★
        </span>
      ))}
      { /* 数値も併記する (スクリーンリーダーや視認性のため) */ }
      <span className="ml-1 text-sm text-gray-600">
        ({clampedRating})
      </span>
    </div>
  );
}
```

画面にはこのように表示されます:

- `rating=4` の場合: ★★★★★ (4) — 黄色い星4つとグレーの星1つ、その横に小さく (4) と数値が表示されます。
 - `rating=null` の場合: グレーの文字で 評価なし と表示されます。
-

1c. BookCard コンポーネント

ファイル: `components/BookCard.tsx`

1冊分の書籍情報をカード形式で表示するコンポーネントです。上で作った `StatusBadge` と `RatingStars` を組み合わせて使います。

```
// =====
// ファイルパス: src/components/BookCard.tsx
// 役割      : 1冊分の書籍をカード形式で表示する。クリックで詳細ページへ遷移。
// 親        : BookList コンポーネントから1冊ずつ渡されて表示される
// -----
// このコンポーネントは、これまでに作った StatusBadge と RatingStars を
// 子として組み合わせて使う「コンポジション（合成）」の好例。
// =====

// (1) 必要なものを import
//     Next.js の Link: クライアントサイド遷移用のリンク要素（フルリロードしない）
//     ./StatusBadge   : 同フォルダの StatusBadge.tsx（拡張子は省略可能）
//     `type BookStatus` : 値ではなく「型」だけを取り込む記法
import Link from "next/link";
import StatusBadge, { type BookStatus } from "./StatusBadge";
import RatingStars from "./RatingStars";

/**
 * BookCard - 書籍1冊分の情報をカード形式で表示するコンポーネント
 *
 * 表示する情報:
 *   - タイトル
 *   - 著者
 *   - 出版社（あれば）
 *   - ステータスバッジ
 *   - 評価（星）
 *
 * カード全体が <Link> で囲まれており、どこをクリックしても /books/{id} へ遷移する。
 */

// (2) Book 型の定義
//     データベースの books テーブルの1行に対応する型。
//     publisher など null OK のカラムは「string | null」のユニオン型で表す。
//     export しているので他ファイルから import { type Book } で使える。
export type Book = {
  id: string;           // UUID（主キー）
  title: string;        // 必須
  author: string;       // 必須
  publisher: string | null; // 任意
  published_date: string | null; // ISO日付文字列（"2024-01-15" など）
  rating: number | null; // 1~5 または null
  status: BookStatus;    // "reading" | "completed" | "want_to_read"
  memo: string | null;   // 感想メモ
  created_at: string;    // 作成日時（自動）
  updated_at: string;    // 更新日時（自動）
};

// (3) Props の型
//     親 (BookList) から book を1つ受け取る、というだけの単純な型。
```

```

type BookCardProps = {
  book: Book;
};

// (4) コンポーネント本体
//   export default で「このファイルの主演」を1つだけ宣言。
//   関数引数 ({ book }: BookCardProps) は分割代入 + 型注釈。
export default function BookCard({ book }: BookCardProps) {
  return (
    // (5) Link はカード全体を「クリック可能なリンク」にする要素
    //   href={`\books/${book.id}`} のテンプレートリテラルで動的にURLを組み立てる
    //   /books/123e4567-e89b-12d3-a456-426614174000 のような遷移先になる
    //
    //   className のクラス文字列の意味 (Tailwind CSS) :
    //     block                ← <Link> (普段はインライン) をブロック要素にする
    //     bg-white             ← 背景色を白に
    //     rounded-lg           ← 角丸 (大きめ)
    //     shadow-md            ← 中ぐらいのドロップシャドウ
    //     hover:shadow-lg      ← マウスを乗せたら影を強くする
    //     transition-shadow    ← 影の変化を滑らかにアニメーション
    //     duration-200         ← アニメーション時間 200ms
    //     p-6                  ← 全方向の内側余白 24px
    //     border               ← 1px の枠線を表示
    //     border-gray-200      ← 枠線の色
    //     hover:border-blue-300 ← ホバー時に枠線の色を変える
    <Link
      href={`\books/${book.id}`}
      className="
        block
        bg-white
        rounded-lg
        shadow-md
        hover:shadow-lg
        transition-shadow
        duration-200
        p-6
        border
        border-gray-200
        hover:border-blue-300
      "
    >
    { /*
      (6) 1行目: タイトル と StatusBadge を左右に配置
      flex                ← フレックスレイアウト
      items-start          ← 上端揃え (タイトルが2行になっても揃う)
      justify-between      ← 左右の端に寄せる
      gap-2                ← 子要素間の隙間
      mb-3                 ← 下方向の外側余白
    */ }
    <div className="flex items-start justify-between gap-2 mb-3">

```

```

    {/*
      line-clamp-2 = テキストが長くて2行で「...」省略する Tailwind 補助クラス
      flex-1      = 余白を最大限取る (バッジに押し出されないようにする)
    */}
    <h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
      {book.title}
    </h2>

    {/*
      (7) 子コンポーネント StatusBadge を呼び出す。Props として status を渡す
      <StatusBadge status="reading" /> のように展開される。
    */}
    <StatusBadge status={book.status} />
  </div>

  {/* (8) 著者名 */}
  <p className="text-sm text-gray-600 mb-1">
    <span className="font-medium text-gray-500">著者:</span>{" "}
    {book.author}
  </p>

  {/*
    (9) 「book.publisher && ...」は短絡評価。
    publisher が null/空 のときは何も表示せず、値があるときだけ <p> を描画。
  */}
  {book.publisher && (
    <p className="text-sm text-gray-600 mb-1">
      <span className="font-medium text-gray-500">出版社:</span>{" "}
      {book.publisher}
    </p>
  )}

  {/* (10) 出版日 (null チェックは publisher と同じパターン) */}
  {book.published_date && (
    <p className="text-sm text-gray-600 mb-3">
      <span className="font-medium text-gray-500">出版日:</span>{" "}
      {book.published_date}
    </p>
  )}

  {/*
    (11) 評価 (星)
    mt-3 pt-3 border-t border-gray-100 で
    「カード本体と評価エリアの間に薄い区切り線を入れる」効果。
  */}
  <div className="mt-3 pt-3 border-t border-gray-100">
    <RatingStars rating={book.rating} />
  </div>

  {/* (12) メモが存在すれば、最大2行までプレビュー表示 */}

```

```

    {book.memo && (
      <p className="mt-2 text-xs text-gray-400 line-clamp-2">
        {book.memo}
      </p>
    )}
  </Link>
);
}

```

1枚のカードはこのように表示されます:

カードは白い背景（ `bg-white` ）に薄いグレーのボーダー（ `border-gray-200` ）と影（ `shadow-md` ）がついた角丸の矩形です。マウスカーソルを乗せると影が濃くなり（ `hover:shadow-lg` ）、ボーダーが薄い青色（ `hover:border-blue-300` ）に変わります。これにより「クリックできる」ことが視覚的に伝わります。

カードの内部構成は上から順に:

1. **1行目（タイトル行）**: 左にタイトルが太字で表示され、右端にステータスバッジが配置されます。タイトルが長い場合は2行まで表示され、それ以降は `...` で省略されます（ `line-clamp-2` ）。
2. **2行目**: 「著者: ○○」のように著者名が小さめのフォントで表示されます。
3. **3行目（出版社がある場合）**: 「出版社: ○○」と表示されます。
4. **4行目（出版日がある場合）**: 「出版日: 2024-01-15」のように表示されます。
5. **区切り線の下**: 薄いグレーの区切り線の下に星評価（例: ★★★★★☆ **(4)**）が表示されます。
6. **最下部（メモがある場合）**: メモの先頭部分が薄いグレーで2行まで表示されます。

カード全体が `<Link>` で囲まれているため、どこをクリックしても `/books/[id]` の詳細ページに遷移します。

1d. BookList コンポーネント

ファイル: `components/BookList.tsx`

BookCard を並べて表示するコンポーネントです。書籍が0冊の場合のメッセージも含みます。

```
// components/BookList.tsx

import BookCard, { type Book } from "../BookCard";

/**
 * BookList - 書籍一覧をグリッドレイアウトで表示するコンポーネント
 *
 * - 書籍が1冊以上ある場合: カードをグリッド状に並べる
 * - 書籍が0冊の場合: 「書籍が登録されていません」メッセージを表示
 *
 * レスポンシブ対応:
 * - スマホ (デフォルト): 1列
 * - タブレット (md): 2列
 * - デスクトップ (lg): 3列
 */

type BookListProps = {
  books: Book[];
};

export default function BookList({ books }: BookListProps) {
  // 書籍が0冊の場合
  if (books.length === 0) {
    return (
      <div className="text-center py-16">
        { /* 大きなアイコン的テキスト */ }
        <p className="text-6xl mb-4">📖</p>
        <h2 className="text-xl font-bold text-gray-700 mb-2">
          書籍が登録されていません
        </h2>
        <p className="text-gray-500 mb-6">
          「新規登録」ボタンから最初の書籍を登録しましょう。
        </p>
        <a
          href="/books/new"
          className="
            inline-block
            bg-blue-600
            text-white
            px-6 py-3
            rounded-lg
            font-medium
            hover:bg-blue-700
            transition-colors
            duration-200
          "
        >
          最初の書籍を登録する
        </a>
      </div>
    );
  }
}
```

```

    </div>
  );
}

// 書籍が1冊以上ある場合
return (
  <div
    className="
      grid
      grid-cols-1
      md:grid-cols-2
      lg:grid-cols-3
      gap-6
    "
  >
    {books.map((book) => (
      <BookCard key={book.id} book={book} />
    ))}
  </div>
);
}

```

表示の説明:

- 書籍がある場合: CSS Grid を使って、画面幅に応じて1～3列にカードが並びます。各カードの間は `gap-6` (24px) の余白があります。
- 書籍がない場合: 画面中央に本の絵文字、「書籍が登録されていません」というメッセージ、そして新規登録ページへのリンクボタンが表示されます。

1e. トップページ

ファイル: `app/page.tsx`

Supabase からデータを取得して BookList に渡すトップページです。Next.js の **Server Component** として実装します。


```
// app/page.tsx

import Link from "next/link";
import { createClient } from "@lib/supabase/server";
import BookList from "@components/BookList";
import { type Book } from "@components/BookCard";

/**
 * トップページ (Server Component)
 *
 * このコンポーネントは Server Component として動作する。
 * つまり、サーバー側で Supabase からデータを取得し、
 * HTML をレンダリングしてからクライアントに送信する。
 *
 * "use client" を書いていない = Server Component (Next.js App Router のデフォルト)
 */
export default async function HomePage() {
  // Supabase クライアントを作成 (サーバー用)
  const supabase = await createClient();

  // books テーブルから全件取得 (作成日の降順=新しい順)
  const { data: books, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  // エラーが発生した場合
  if (error) {
    console.error("書籍の取得に失敗しました:", error.message);
    // エラー時でも画面は表示する (空の一覧として)
    // 本番アプリでは error boundary を使うことも検討する
  }

  // data が null の場合は空配列にする
  const bookList: Book[] = books ?? [];

  return (
    <div className="min-h-screen bg-gray-50">
      { /* ===== ヘッダー ===== */ }
      <header className="bg-white shadow-sm border-b border-gray-200">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center justify-between">
            { /* アプリタイトル */ }
            <div>
              <h1 className="text-2xl sm:text-3xl font-bold text-gray-900">
                書籍管理アプリ
              </h1>
              <p className="mt-1 text-sm text-gray-500">
                あなたの読書記録を管理しましょう
              </p>
            </div>
          </div>
        </div>
      </header>
    </div>
  );
}
```

```

    </p>
  </div>

  {/* 新規登録ボタン */}
  <Link
    href="/books/new"
    className="
      inline-flex items-center gap-2
      bg-blue-600
      text-white
      px-4 py-2.5
      rounded-lg
      font-medium
      text-sm
      hover:bg-blue-700
      transition-colors
      duration-200
      shadow-sm
    "
  >
    <svg
      className="w-5 h-5"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        d="M12 4v16m8-8H4"
      />
    </svg>
    新規登録
  </Link>
</div>
</div>
</header>

{/* ===== メインコンテンツ ===== */}
<main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
  {/* 書籍数の表示 */}
  <div className="mb-6">
    <p className="text-sm text-gray-600">
      全 <span className="font-bold text-gray-900">{bookList.length}</span> 冊
    </p>
  </div>

  {/* 書籍一覧 */}
  <BookList books={bookList} />

```

```

    </main>
  </div>
);
}

```

画面全体はこのように表示されます:

画面は大きく **ヘッダー** と **メインコンテンツ** の2つの領域に分かれています。背景は薄いグレー（ `bg-gray-50` ）です。

ヘッダー部分: - 白い背景（ `bg-white` ）に薄い影（ `shadow-sm` ）と下ボーダーがついた帯状のヘッダーです。 - 左側に「書籍管理アプリ」というアプリタイトルが大きな太字で表示され、その下に「あなたの読書記録を管理しましょう」という小さな説明文があります。 - 右側に青いボタン「+ 新規登録」が配置されています。ボタンの左にはプラスアイコン（SVG）が表示されます。 - ヘッダーの内容は `max-w-7xl`（1280px）で中央揃えされ、画面幅が広くても広がりすぎません。

メインコンテンツ部分: - ヘッダーの下に「全 N 冊」という書籍数のテキストがあります。数字部分は太字です。 - その下に `BookList` コンポーネントが配置されます。 - **書籍が登録されている場合:** 白いカードがグリッド状に並びます。スマホでは1列、タブレットでは2列、デスクトップでは3列です。各カードの間には適度な余白があります。 - **書籍が0冊の場合:** 画面中央に本の絵文字と「書籍が登録されていません」というメッセージ、そして「最初の書籍を登録する」ボタンが表示されます。

例として、3冊の書籍が登録されている場合のデスクトップ表示をイメージすると:



1f. データ取得フローの図解

以下は、トップページが表示されるまでのデータの流れを示す Mermaid シーケンス図です。



ポイント:

- Server Component はサーバー側で実行されるため、Supabase へのリクエストもサーバーから行われます。クライアント（ブラウザ）から直接データベースにアクセスすることはありません。
- ブラウザは完成した HTML を受け取って表示するだけなので、初回表示が高速です。
- Supabase のシークレットキーがブラウザに漏れることもありません（サーバー側でのみ使用されるため）。

2. ローディング状態

Next.js App Router では、`loading.tsx` ファイルを作成すると、ページの読み込み中に自動的にローディング表示を行います。

LoadingSpinner コンポーネント

ファイル: `components>LoadingSpinner.tsx`

```
// components/LoadingSpinner.tsx
"use client";

/**
 * LoadingSpinner - ローディング中に表示するスピナーコンポーネント
 *
 * CSS アニメーション (animate-spin) で回転する円を表示する。
 * "use client" を付けているのは、アニメーションがクライアント側で
 * 実行されるため (厳密には Server Component でも動作するが、
 * 再利用性のために Client Component にしている)。
 */

type LoadingSpinnerProps = {
  /** スピナーのサイズ (デフォルト: "md") */
  size?: "sm" | "md" | "lg";
  /** スピナーの下に表示するテキスト */
  message?: string;
};

export default function LoadingSpinner({
  size = "md",
  message = "読み込み中...",
}: LoadingSpinnerProps) {
  // サイズに応じたクラスを定義
  const sizeClasses = {
    sm: "w-6 h-6 border-2",
    md: "w-10 h-10 border-3",
    lg: "w-16 h-16 border-4",
  };

  return (
    <div className="flex flex-col items-center justify-center py-16">
      {/* 回転するスピナー */}
      <div
        className={`
          ${sizeClasses[size]}
          border-gray-300
          border-t-blue-600
          rounded-full
          animate-spin
        `}
        role="status"
        aria-label="読み込み中"
      />
      {/* メッセージ */}
      {message && (
        <p className="mt-4 text-sm text-gray-500">
          {message}
        </p>
      )}
    </div>
  );
}
```

```
    })  
  </div>  
);  
}
```

loading.tsx

ファイル: `app/loading.tsx`

```
// app/loading.tsx

import LoadingSpinner from "@/components/LoadingSpinner";

/**
 * ルートレイアウトのローディング UI
 *
 * Next.js App Router は、ページコンポーネント (page.tsx) の
 * データ取得中にこの loading.tsx を自動的に表示する。
 *
 * 仕組み:
 * 1. ユーザーがページに遷移する
 * 2. page.tsx 内の async 処理 (Supabase からのデータ取得等) が実行される
 * 3. その間、この loading.tsx が表示される
 * 4. データ取得が完了すると、page.tsx の内容に自動的に切り替わる
 *
 * これは React の Suspense を内部的に利用している。
 */
export default function Loading() {
  return (
    <div className="min-h-screen bg-gray-50">
      { /* ヘッダーのスケルトン (ローディング中もヘッダーのような見た目を維持) */ }
      <header className="bg-white shadow-sm border-b border-gray-200">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center justify-between">
            <div>
              <div className="h-8 w-48 bg-gray-200 rounded animate-pulse" />
              <div className="mt-2 h-4 w-64 bg-gray-100 rounded animate-pulse" />
            </div>
            <div className="h-10 w-28 bg-gray-200 rounded-lg animate-pulse" />
          </div>
        </div>
      </header>

      { /* メインコンテンツのローディング表示 */ }
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <LoadingSpinner size="lg" message="書籍データを読み込んでいます..." />
      </main>
    </div>
  );
}
```

表示の説明:

ローディング中は、ヘッダー部分がスケルトン UI（灰色の長方形がパルスアニメーションする）として表示され、メインコンテンツ領域の中央に大きなスピナー（回転する円）と「書籍データを読み込んでいます...」というメッセージが表示されます。ユーザーは「何かが読み込まれている」ことをすぐに理解できます。

3. エラーハンドリング

ファイル: `app/error.tsx`

ページのレンダリング中にエラーが発生した場合に表示される画面です。Next.js App Router の Error Boundary 機能を利用しています。

```
// app/error.tsx
"use client";

/**
 * エラーページ
 *
 * 重要: error.tsx は必ず "use client" でなければならない。
 * これは Next.js の仕様で、Error Boundary は Client Component である必要がある。
 *
 * このコンポーネントは以下の場合に自動的に表示される:
 * - Server Component でエラーが throw された場合
 * - Client Component で未処理のエラーが発生した場合
 * - Supabase からのデータ取得が失敗した場合 (throw した場合)
 */

type ErrorPageProps = {
  /** 発生したエラーオブジェクト */
  error: Error & { digest?: string };
  /** エラーからの復帰を試みる関数 (ページの再レンダリングを試行する) */
  reset: () => void;
};

export default function ErrorPage({ error, reset }: ErrorPageProps) {
  return (
    <div className="min-h-screen bg-gray-50 flex items-center justify-center px-4">
      <div className="max-w-md w-full bg-white rounded-lg shadow-lg p-8 text-center">
        {/* エラーアイコン */}
        <div className="mx-auto w-16 h-16 bg-red-100 rounded-full flex items-center justify-center">
          <svg
            className="w-8 h-8 text-red-600"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24">
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-2.5L13.732 4c-.768-.768-1.934-.768-2.704 0z"
            />
          </svg>
        </div>

        {/* エラーメッセージ */}
        <h2 className="text-xl font-bold text-gray-900 mb-2">
          エラーが発生しました
        </h2>
        <p className="text-gray-600 mb-6">
          申し訳ありません。データの取得中にエラーが発生しました。
        </p>
      </div>
    </div>
  );
}
```

```

        しばらく時間をおいてから再度お試しください。
    </p>

    {/* エラー詳細（開発時のデバッグ用） */}
    {process.env.NODE_ENV === "development" && (
        <details className="mb-6 text-left">
            <summary className="text-sm text-gray-500 cursor-pointer hover:text-gray-700">
                エラー詳細を表示
            </summary>
            <pre className="mt-2 p-3 bg-gray-100 rounded text-xs text-red-600 overflow-auto">
                {error.message}
            </pre>
        </details>
    )}

    {/* アクションボタン */}
    <div className="flex flex-col sm:flex-row gap-3 justify-center">
        <button
            onClick={reset}
            className="
                px-6 py-2.5
                bg-blue-600
                text-white
                rounded-lg
                font-medium
                hover:bg-blue-700
                transition-colors
                duration-200
            "
        >
            もう一度試す
        </button>
        <a
            href="/"
            className="
                px-6 py-2.5
                bg-gray-100
                text-gray-700
                rounded-lg
                font-medium
                hover:bg-gray-200
                transition-colors
                duration-200
            "
        >
            トップに戻る
        </a>
    </div>
</div>
</div>

```

```
);  
}
```

表示の説明:

画面中央に白いカードが表示されます。赤い三角形の警告アイコン、「エラーが発生しました」というメッセージ、そして「もう一度試す」ボタンと「トップに戻る」ボタンが配置されます。開発環境（`NODE_ENV === "development"`）では、エラーの詳細メッセージも折りたたみで確認できます。

4. 登録機能の実装

ここからは書籍の新規登録機能を作ります。ユーザーがフォームに情報を入力し、Supabase のデータベースに新しいレコードを INSERT する流れです。

4a. BookForm コンポーネント

ファイル: `components/BookForm.tsx`

フォーム全体を管理するコンポーネントです。新規登録と編集の両方で使えるように設計します（この章では新規登録のみ使用、編集は次章で使用）。

```
// components/BookForm.tsx
"use client";

/**
 * BookForm - 書籍の登録・編集フォームコンポーネント
 *
 * "use client" が必要な理由:
 * - useState でフォームの入力状態を管理する
 * - onChange イベントハンドラを使う
 * - フォーム送信時の処理 (onSubmit) をハンドリングする
 * これらはすべてブラウザ側で動作する機能のため、Client Component にする。
 *
 * このコンポーネントは「登録」と「編集」の両方で使い回す。
 * - 新規登録時: initialData を渡さない → 空のフォーム
 * - 編集時: initialData に既存データを渡す → 値が入ったフォーム
 */

import { useState, type FormEvent } from "react";
import { type BookStatus } from "../StatusBadge";

// フォームのデータ型 (データベースの books テーブルに対応するが、
// id, created_at, updated_at はフォームでは扱わない)
export type BookFormData = {
  title: string;
  author: string;
  publisher: string;
  published_date: string;
  rating: number | null;
  status: BookStatus;
  memo: string;
};

// Props の型定義
type BookFormProps = {
  /** 編集時に渡す初期データ (新規登録時は undefined) */
  initialData?: BookFormData;
  /** フォーム送信時に呼ばれるコールバック関数 */
  onSubmit: (data: BookFormData) => Promise<void>;
  /** 送信ボタンのテキスト ("登録する" / "更新する" など) */
  submitLabel?: string;
  /** 送信中かどうか (ボタンの無効化・ローディング表示に使う) */
  isSubmitting?: boolean;
};

// フォームの初期値 (新規登録時に使う)
const defaultFormData: BookFormData = {
  title: "",
  author: "",
  publisher: "",

```

```

    published_date: "",
    rating: null,
    status: "want_to_read",
    memo: "",
  };

export default function BookForm({
  initialData,
  onSubmit,
  submitLabel = "登録する",
  isSubmitting = false,
}: BookFormProps) {
  // ----- State -----
  // フォームの入力値を管理する state
  // initialData が渡されていればそれを使い、なければデフォルト値を使う
  const [formData, setFormData] = useState<BookFormData>(
    initialData ?? defaultFormData
  );

  // バリデーションエラーを管理する state
  // キーがフィールド名、値がエラーメッセージ
  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>({});

  // ----- バリデーション -----
  /**
   * フォーム全体のバリデーションを行う
   * @returns バリデーションを通過したら true、エラーがあれば false
   */
  const validate = (): boolean => {
    const newErrors: Partial<Record<keyof BookFormData, string>> = {};

    // タイトル: 必須、100文字以内
    if (!formData.title.trim()) {
      newErrors.title = "タイトルは必須です";
    } else if (formData.title.trim().length > 100) {
      newErrors.title = "タイトルは100文字以内で入力してください";
    }

    // 著者: 必須、50文字以内
    if (!formData.author.trim()) {
      newErrors.author = "著者は必須です";
    } else if (formData.author.trim().length > 50) {
      newErrors.author = "著者は50文字以内で入力してください";
    }

    // 出版社: 任意、50文字以内
    if (formData.publisher.trim().length > 50) {
      newErrors.publisher = "出版社は50文字以内で入力してください";
    }
  };

```

```

// メモ: 任意、1000文字以内
if (formData.memo.trim().length > 1000) {
  newErrors.memo = "メモは1000文字以内で入力してください";
}

setErrors(newErrors);

// エラーが1つもなければ true を返す
return Object.keys(newErrors).length === 0;
};

// ----- イベントハンドラ -----

/**
 * テキスト入力フィールドの変更ハンドラ
 * input, textarea, select の onChange に共通で使える
 *
 * -----
 * (a) 引数 e は「変更イベントオブジェクト」。ユーザーが文字を打つと
 *     React がこのオブジェクトをハンドラに渡してくる。
 * (b) e.target は「変更が起きた DOM 要素」。ここから name と value を取り出す。
 *     - name : <input name="title"> のように JSX で指定した名前
 *     - value : 現在の入力値 (文字列)
 * -----
 */
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>
) => {
  // (1) e.target から name, value を分割代入で取り出す
  //     例: <input name="title" value="..." /> なら
  //         name = "title", value = ユーザーが打った文字列
  const { name, value } = e.target;

  // (2) 現在の formData を「コピーして1つの項目だけ書き換える」更新パターン
  //     setData((prev) => ...) の prev は「直前の formData の値」。
  //     ...prev は「prev の中身を全部コピー」(スプレッド構文)。
  //     [name]: value は「name 変数の値をキーにして value を入れる」記法
  //     (Computed Property Names)。
  //
  //     例: name="title", value="React入門" のとき
  //         { ...prev, title: "React入門" } という新しいオブジェクトを作る。
  //     React は「formData が新しいオブジェクトに入れ替わった」と検知して
  //     再描画する。直接 prev.title = value のように書き換えてはいけない。
  setData((prev) => ({
    ...prev,
    [name]: value,
  }));

  // (3) ユーザーが入力をやり直し始めたら、そのフィールドの古いエラーを消す。
  //     [name]: undefined を入れることでエラー表示が消える。

```

```

//   `as keyof BookFormData` は「name は BookFormData のキーですよ」と
//   TS に教えるアサーション（型注釈の補助）。
if (errors[name as keyof BookFormData]) {
  setErrors((prev) => ({
    ...prev,
    [name]: undefined,
  }));
}
};

/**
 * 評価 (rating) の変更ハンドラ
 * select の値は文字列なので、数値に変換する必要がある
 */
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,
    rating: value === "" ? null : Number(value),
  }));
};

/**
 * フォーム送信ハンドラ
 * <form onSubmit={handleSubmit}> から呼ばれる。
 * - 引数 e はフォーム送信イベントオブジェクト。
 * - 関数全体に async が付くので、内部で await が使える。
 */
const handleSubmit = async (e: FormEvent) => {
  // (1) e.preventDefault() を呼ばないと、ブラウザが既定の動作として
  //     ページ全体をリロードしてしまう (HTML 本来の挙動)。
  //     SPA では絶対に必要な「お決まりの一行」。
  e.preventDefault();

  // (2) バリデーション関数を呼び、エラーがあれば中断
  //     validate() は内部で setErrors を呼ぶので、画面にエラー文も表示される。
  if (!validate()) {
    return; // 関数を終了 (送信しない)
  }

  // (3) 親コンポーネントから渡された onSubmit コールバックを呼び
  //     onSubmit は Promise を返す async 関数なので await で待つ。
  //     ここで実際のDB書き込みやページ遷移が行われる (実装は親側にある)。
  await onSubmit(formData);
};

// ----- 表示 -----
return (
  <form onSubmit={handleSubmit} className="space-y-6">
    {/* ===== タイトル ===== */}

```



```

<div>
  <label
    htmlFor="title"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    タイトル <span className="text-red-500">*</span>
  </label>
  <input
    type="text"
    id="title"
    name="title"
    value={formData.title}
    onChange={handleChange}
    placeholder="例: リーダブルコード"
    className={`
      w-full px-4 py-2.5
      border rounded-lg
      text-gray-900
      placeholder-gray-400
      focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
      transition-colors duration-200
      ${errors.title ? "border-red-500 bg-red-50" : "border-gray-300"}
    `}
  />
  {errors.title && (
    <p className="mt-1 text-sm text-red-600">{errors.title}</p>
  )}
</div>

{ /* ===== 著者 ===== */ }

<div>
  <label
    htmlFor="author"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    著者 <span className="text-red-500">*</span>
  </label>
  <input
    type="text"
    id="author"
    name="author"
    value={formData.author}
    onChange={handleChange}
    placeholder="例: Dustin Boswell"
    className={`
      w-full px-4 py-2.5
      border rounded-lg
      text-gray-900
      placeholder-gray-400
      focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent

```



```

name="published_date"
value={formData.published_date}
onChange={handleChange}
className="
  w-full px-4 py-2.5
  border border-gray-300 rounded-lg
  text-gray-900
  focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
  transition-colors duration-200
"
/>
</div>

{/* ===== 評価 と ステータス を横に並べる ===== */}
<div className="grid grid-cols-1 sm:grid-cols-2 gap-6">
  {/* 評価 */}
  <div>
    <label
      htmlFor="rating"
      className="block text-sm font-medium text-gray-700 mb-1"
    >
      評価
    </label>
    <select
      id="rating"
      name="rating"
      value={formData.rating ?? ""}
      onChange={handleRatingChange}
      className="
        w-full px-4 py-2.5
        border border-gray-300 rounded-lg
        text-gray-900
        focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
        transition-colors duration-200
        bg-white
      "
    >
      <option value="">選択してください</option>
      <option value="1">★☆☆☆☆ (1)</option>
      <option value="2">★★☆☆☆ (2)</option>
      <option value="3">★★★☆☆ (3)</option>
      <option value="4">★★★★☆ (4)</option>
      <option value="5">★★★★★ (5)</option>
    </select>
  </div>

  {/* ステータス */}
  <div>
    <label
      htmlFor="status"

```

```

        className="block text-sm font-medium text-gray-700 mb-1"
      >
        ステータス <span className="text-red-500">*</span>
      </label>
      <select
        id="status"
        name="status"
        value={formData.status}
        onChange={handleChange}
        className="
          w-full px-4 py-2.5
          border border-gray-300 rounded-lg
          text-gray-900
          focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
          transition-colors duration-200
          bg-white
        "
      >
        <option value="want_to_read">読みたい</option>
        <option value="reading">読書中</option>
        <option value="completed">読了</option>
      </select>
    </div>
  </div>

  {/* ===== メモ ===== */}
  <div>
    <label
      htmlFor="memo"
      className="block text-sm font-medium text-gray-700 mb-1"
    >
      メモ
    </label>
    <textarea
      id="memo"
      name="memo"
      value={formData.memo}
      onChange={handleChange}
      rows={4}
      placeholder="この書籍に関するメモを自由に入力してください"
      className={`
        w-full px-4 py-2.5
        border rounded-lg
        text-gray-900
        placeholder-gray-400
        focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
        transition-colors duration-200
        resize-vertical
        ${errors.memo ? "border-red-500 bg-red-50" : "border-gray-300"}
      `}
    >

```

```

/>
{errors.memo && (
  <p className="mt-1 text-sm text-red-600">{errors.memo}</p>
)}
<p className="mt-1 text-xs text-gray-400">
  {formData.memo.length} / 1000 文字
</p>
</div>

{/* ===== 送信ボタン ===== */}
<div className="flex items-center gap-4 pt-4">
  <button
    type="submit"
    disabled={isSubmitting}
    className={`
      px-8 py-3
      rounded-lg
      font-medium
      text-white
      transition-colors duration-200
      ${
        isSubmitting
          ? "bg-blue-400 cursor-not-allowed"
          : "bg-blue-600 hover:bg-blue-700"
      }
    `}
  >
    {isSubmitting ? (
      <span className="flex items-center gap-2">
        {/* 送信中のスピナー */}
        <svg
          className="animate-spin h-5 w-5"
          viewBox="0 0 24 24"
          fill="none"
        >
          <circle
            className="opacity-25"
            cx="12"
            cy="12"
            r="10"
            stroke="currentColor"
            strokeWidth="4"
          />
          <path
            className="opacity-75"
            fill="currentColor"
            d="M4 12a8 8 0 018-8v4a4 4 0 04 4H4z"
          />
        </svg>
        送信中...
      )
    )}
  </div>

```

```

        </span>
    ) : (
        submitLabel
    )}
</button>

{/* キャンセルボタン（トップページに戻る） */}
<a
    href="/"
    className="
        px-6 py-3
        rounded-lg
        font-medium
        text-gray-700
        bg-gray-100
        hover:bg-gray-200
        transition-colors duration-200
    "
    >
        キャンセル
    </a>
</div>
</form>
);
}

```

フォームはこのように表示されます:

白い背景のページ上に、縦に並んだフォームフィールドが表示されます。各フィールドの構成は以下の通りです。

新規書籍を登録

タイトル *

例: リーダブルコード

著者 *

例: Dustin Boswell

出版社

例: オライリージャパン

出版日

yyyy/mm/dd

評価

選択してください ▼

ステータス *

読みたい ▼

メモ

この書籍に関するメモを自由に入力してください

0 / 1000 文字

登録する

キャンセル

- 各フィールドのラベルの横にある * マーク（赤色）は必須フィールドを示します。
- 入力フィールドにフォーカスすると、青い枠線（`focus:ring-2 focus:ring-blue-500`）が表示されます。
- バリデーションエラーがある場合、そのフィールドの枠線が赤くなり（`border-red-500`）、背景が薄い赤色（`bg-red-50`）になり、フィールドの下にエラーメッセージが赤文字で表示されます。
- 「評価」と「ステータス」はデスクトップでは横に並び（`sm:grid-cols-2`）、スマホでは縦に並びます。
- メモ欄の下には入力文字数カウンター（`0 / 1000 文字`）が表示されます。
- 送信ボタンは青色で、送信中は薄い青色になりスピナーアイコンが回転します。

4b. 登録ページ

ファイル: `app/books/new/page.tsx`

BookForm を使って新規登録画面を構成するページです。


```
// app/books/new/page.tsx
"use client";

/**
 * 書籍登録ページ
 *
 * "use client" を付けている理由:
 * - BookForm (Client Component) の onSubmit コールバックで
 *   Supabase への INSERT 処理を行う
 * - useRouter で登録後のリダイレクトを行う
 * - useState で送信中の状態を管理する
 *
 * ※ Server Action を使う方法もあるが、この章では
 *   初心者にとって分かりやすい Client Component 方式で実装する。
 */

import { useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";
import BookForm, { type BookFormData } from "@/components/BookForm";

export default function NewBookPage() {
  const router = useRouter();

  // 送信中かどうかの状態
  const [isSubmitting, setIsSubmitting] = useState(false);

  // エラーメッセージの状態
  const [submitError, setSubmitError] = useState<string | null>(null);

  /**
   * フォーム送信時の処理
   * BookForm の onSubmit に渡すコールバック関数
   */
  const handleSubmit = async (data: BookFormData) => {
    // (1) 「送信中フラグ」を ON にする → ボタンが「送信中…」表示になり無効化される
    //      setSubmitError(null) で前回のエラーメッセージをクリアする
    setIsSubmitting(true);
    setSubmitError(null);

    // (2) try / catch / finally の3ブロック構成
    //      - try      : 通常実行する処理。途中で例外 (Error) が起きると catch へ飛ぶ
    //      - catch    : 想定外のエラーをキャッチ
    //      - finally  : 成功・失敗どちらでも最後に必ず実行される
    try {
      // (3) Supabase クライアントを作る
      //      ここではブラウザ用クライアント (ブラウザのJSから直接DBにつなぐ)。
      //      RLSポリシーがあるので、許可された操作だけが通る。
      const supabase = createClient();

```

```

// (4) books テーブルに INSERT する
//   .from("books") : 操作対象テーブルを指定
//   .insert({...}) : 書き込みたいレコードのオブジェクト
//   await           : 非同期処理を待ち、結果を { error } に展開
//
//   入力ガード:
//     data.title.trim()           ← 前後の空白を削る
//     data.publisher.trim() || null ← 空文字列なら null (DBではnullで保存)
//     data.published_date || null  ← 同上
//     data.memo.trim() || null     ← 同上
//   こうすることで「空欄」と「null」を区別せずに DB に綺麗な値が入る。
const { error } = await supabase.from("books").insert({
  title: data.title.trim(),
  author: data.author.trim(),
  publisher: data.publisher.trim() || null,
  published_date: data.published_date || null,
  rating: data.rating,
  status: data.status,
  memo: data.memo.trim() || null,
});

// (5) Supabase が返す error は「成功なら null、失敗ならオブジェクト」。
//   エラーがあれば、ユーザー向けメッセージをセットして関数を終了。
if (error) {
  console.error("書籍の登録に失敗しました:", error.message);
  setSubmitError(
    "書籍の登録に失敗しました。入力内容を確認して再度お試しください。"
  );
  return;
}

// (6) 登録成功 → トップページに戻る
//   router.push("/") : URLを "/" に変える (クライアント側遷移)
//   ページ全体のリロードは起きないので体感が速い
router.push("/");

// (7) router.refresh() で Server Component を再実行させ、
//   登録したばかりのデータも含む最新の一覧を取得・表示する。
router.refresh();
} catch (err) {
  // (8) 想定外のエラー (ネットワーク切断、JSON パース失敗など) の最後の砦
  console.error("予期しないエラー:", err);
  setSubmitError("予期しないエラーが発生しました。");
} finally {
  // (9) 成功・失敗いずれでもボタンの「送信中」状態は解除する
  setIsSubmitting(false);
}
};

```



```

    { /* エラーメッセージ（送信失敗時に表示） */ }
    { submitError && (
      <div className="mb-6 p-4 bg-red-50 border border-red-200 rounded-lg">
        <div className="flex items-center gap-2">
          <svg
            className="w-5 h-5 text-red-600 flex-shrink-0"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M12 8v4m0 4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z"
            />
          </svg>
          <p className="text-sm text-red-700">{submitError}</p>
        </div>
      </div>
    ) }

    { /* フォーム本体 */ }
    <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6 sm:p-8">
      <BookForm
        onSubmit={handleSubmit}
        submitLabel="登録する"
        isSubmitting={isSubmitting}
      />
    </div>
  </main>
</div>
);
}

```

「登録ボタンを押すとこうなる」の説明:

1. ユーザーがフォームの各フィールドに情報を入力し、「登録する」ボタンをクリックします。
2. まず **クライアント側でバリデーション** が行われます。 - タイトルと著者が空の場合、赤いエラーメッセージ（「タイトルは必須です」等）が該当フィールドの下に表示され、送信は行われません。 - 文字数制限を超えている場合も同様です。
3. バリデーションに通ると、「登録する」ボタンが **薄い青色に変わり**、スピナー（回転アイコン）と「送信中...」というテキストに変化します。ボタンは無効化（ **disabled** ）されるため、二重送信を防止します。
4. Supabase への INSERT リクエストが **ブラウザから直接** Supabase API に送信されます。
5. **成功した場合**: `router.push("/")` によりトップページにリダイレクトされます。一覧が再読み込みされ、今登録した書籍がリストの先頭に表示されます。画面遷移はスムーズで、ページ全体がリロードされることはありません。
6. **失敗した場合**: フォームの上部に赤い背景のエラーメッセージ（「書籍の登録に失敗しました。入力内容を確認して再度お試しください。」）が表示されます。フォームの入力内容は保持されるので、修正して再送信できます。

4c. 登録フローの図解



5. コード解説

ここまでの実装したコードの重要なポイントを、初心者向けに詳しく解説します。

5-1. Server Component と Client Component の使い分け

この章で作ったコンポーネントは、2種類に分かれています。

コンポーネント	種類	理由
<code>app/page.tsx</code>	Server Component	Supabase からのデータ取得をサーバー側で行うため
<code>StatusBadge</code>	Server Component	状態を持たない純粋な表示コンポーネントのため
<code>RatingStars</code>	Server Component	同上
<code>BookCard</code>	Server Component	同上
<code>BookList</code>	Server Component	同上
<code>LoadingSpinner</code>	Client Component	アニメーションの再利用性のため (<code>"use client"</code>)
<code>app/error.tsx</code>	Client Component	Next.js の仕様上必須 (<code>"use client"</code>)
<code>BookForm</code>	Client Component	<code>useState</code> , <code>onChange</code> 等を使うため (<code>"use client"</code>)
<code>app/books/new/page.tsx</code>	Client Component	<code>useRouter</code> , <code>useState</code> を使うため (<code>"use client"</code>)

なぜこの使い分けが重要なのか:

- **Server Component のメリット:** JavaScript バンドルに含まれないため、ブラウザに送信されるデータが少なく、ページの読み込みが速くなります。また、サーバー上で直接データベースにアクセスできるため、API エンドポイントを別途作る必要がありません。
- **Client Component のメリット:** `useState`、`useEffect`、イベントハンドラなどの React のインタラクティブ機能を使えます。ユーザーの操作に応じて画面を動的に更新できます。

初心者がつまづきやすいポイント:

「`"use client"` をどこに書くか迷う」 — 原則として、`useState` や `useEffect`、`onClick` などのイベントハンドラを使うコンポーネントにだけ `"use client"` を付けます。書かなければ Server Component になります (App Router のデフォルト)。「迷ったら Server Component のままにして、エラーが出たら `"use client"` を追加する」というアプローチでも構いません。

5-2. statusConfig のパターン (StatusBadge)

```
const statusConfig: Record<BookStatus, { label: string; bgColor: string; textColor: string }> = {
  reading: { label: "読書中", bgColor: "bg-blue-100", textColor: "text-blue-800" },
  // ...
};
```

なぜ if 文ではなくオブジェクトを使うのか:

`if (status === "reading") { ... } else if (status === "completed") { ... }` と書くこともできますが、オブジェクトで管理する方が:

1. コードが短くなる: 条件分岐を書かなくてよい。
2. 拡張しやすい: 新しいステータスを追加するときにオブジェクトに1行追加するだけ。
3. 型安全: `Record<BookStatus, ...>` を使うことで、全てのステータスに対する設定を定義し忘れるとTypeScriptがエラーを出す。

5-3. `line-clamp-2` について (BookCard)

```
<h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
```

`line-clamp-2` は Tailwind CSS のユーティリティクラスで、テキストを2行までに制限し、それを超える部分を `...` で省略します。書籍のタイトルが長い場合にカードのレイアウトが崩れるのを防ぎます。

5-4. Book 型と Supabase のテーブル構造の対応

```
export type Book = {  
  id: string;  
  title: string;  
  author: string;  
  publisher: string | null;  
  // ...  
};
```

なぜ `publisher` が `string | null` なのか:

データベースの `publisher` カラムは NULL を許容しています（任意入力項目だから）。TypeScript では `null` を明示的に型に含める必要があるため、`string | null` としています。

初心者がつまづきやすいポイント:

null と undefined の違い — データベースから取得した値が「存在しない」場合は `null` が返ります。一方、JavaScript のオブジェクトで「プロパティ自体が存在しない」場合は `undefined` になります。Supabase から返ってくるデータは常に `null` (`undefined` ではない) なので、型定義は `string | null` とします。

5-5. フォームの状態管理 (BookForm)

```
const [formData, setFormData] = useState<BookFormData>(initialData ?? defaultFormData);
```

なぜ1つの state にまとめるのか:

各フィールドごとに別々の `useState` を使う方法もあります:

```
// NG ではないが、フィールド数が多いと管理が大変
const [title, setTitle] = useState("");
const [author, setAuthor] = useState("");
const [publisher, setPublisher] = useState("");
// ...7つの useState...
```

これだと、フィールドが増えるたびに `useState` が増え、`handleChange` も個別に書く必要があります。1つのオブジェクトにまとめることで:

1. コードが簡潔: `handleChange` を全フィールドで共有できる。
2. データの受け渡しが楽: `onSubmit(formData)` で全フィールドを一度に渡せる。
3. 初期値の設定が楽: `initialData ?? defaultFormData` で一括設定できる（編集時に便利）。

5-6. `router.push` と `router.refresh` の組み合わせ（NewBookPage）

```
router.push("/");
router.refresh();
```

なぜ両方必要なのか:

- `router.push("/")` はトップページに遷移しますが、Next.js はパフォーマンスのためにページの内容をキャッシュしています。キャッシュが残っていると、登録前の古いデータが表示されることがあります。
- `router.refresh()` はキャッシュを無効化し、Server Component を再実行させます。これにより、Supabase から最新のデータが取得され、新しく登録した書籍が一覧に反映されます。

5-7. 空文字列を `null` に変換する理由

```
publisher: data.publisher.trim() || null,
```

なぜ空文字列のまま保存しないのか:

- データベースでは「値がない」ことを `NULL` で表現するのが一般的です。
- 空文字列 `""` と `NULL` は意味が異なります。`""` は「入力したが空だった」、`NULL` は「入力されていない」を意味します。
- 将来的に「出版社が未入力の書籍」を検索する場合、`WHERE publisher IS NULL` で統一的に取得できます。`""` が混在すると `WHERE publisher IS NULL OR publisher = ''` と書く必要があり面倒です。
- `data.publisher.trim() || null` は「空白を除去した後に空文字列なら `null` にする」という意味です。`||` 演算子は左辺が falsy（空文字列は falsy）なら右辺を返します。

5-8. e.preventDefault() の役割

```
const handleSubmit = async (e: FormEvent) => {
  e.preventDefault();
  // ...
};
```

HTML の `<form>` は、送信ボタンが押されるとデフォルトでページ全体をリロードしてデータを送信しようとします。

`e.preventDefault()` はこのデフォルト動作を止め、JavaScript で非同期にデータを送信できるようにします。これを忘れるとページがリロードされ、`useState` の値がすべてリセットされてしまいます。

6. 動作確認手順

以下の手順で、実装した機能が正しく動作するか確認しましょう。

前提

- 開発サーバーが起動していること（`npm run dev` を実行済み）
- Supabase のプロジェクトとデータベースが前章で構築済みであること
- `.env.local` に Supabase の接続情報が設定されていること

ステップ1: トップページが表示確認（書籍0冊の状態）

- ブラウザで `http://localhost:3000` を開きます。
- 以下を確認してください: - ヘッダーに「書籍管理アプリ」と表示されている - 「+ 新規登録」ボタンが右上に表示されている - 「全 0 冊」と表示されている - 「書籍が登録されていません」というメッセージと「最初の書籍を登録する」ボタンが中央に表示されている
- ブラウザの開発者ツール（F12）の Console タブにエラーが出ていないことを確認します。

ステップ2: 登録ページへの遷移確認

- ヘッダーの「+ 新規登録」ボタンをクリックします。
- `http://localhost:3000/books/new` に遷移し、登録フォームが表示されることを確認します。
- 以下の要素が表示されていることを確認: - 左上に戻るボタン（矢印アイコン） - 「書籍を登録する」というタイトル - タイトル、著者、出版社、出版日、評価、ステータス、メモの各フィールド - 「登録する」と「キャンセル」ボタン

ステップ3: バリデーションの確認

- 何も入力せずに「登録する」ボタンをクリックします。
- 「タイトルは必須です」「著者は必須です」というエラーメッセージが赤文字で表示されることを確認します。
- タイトルと著者のフィールドの枠線が赤くなっていることを確認します。

4. タイトルフィールドに何か入力すると、タイトルのエラーメッセージが消えることを確認します。

ステップ4: 書籍の登録

1. 以下のテスト用データを入力します: - タイトル: **リーダブルコード** - 著者: **Dustin Boswell** - 出版社: **オライリージャパン** - 出版日: **2012-06-23** - 評価: **★★★★☆ (4)** - ステータス: **読了** - メモ: **読みやすいコードを書くための実践的なテクニック集。**
2. 「登録する」ボタンをクリックします。
3. ボタンが一瞬「送信中...」に変わることを確認します。
4. トップページ (<http://localhost:3000>) にリダイレクトされることを確認します。
5. 「全 1 冊」と表示され、登録した書籍のカードが表示されることを確認します。

ステップ5: カード表示の確認

1. 表示されたカードに以下の情報があることを確認: - タイトル「リーダブルコード」 - ステータスバッジ「読了」（緑色） - 著者「Dustin Boswell」 - 出版社「オライリージャパン」 - 出版日「2012-06-23」 - 星評価「★★★★☆ (4)」 - メモ「読みやすいコードを書くための実践的なテクニック集。」
2. カードにマウスを乗せると影が変化することを確認します。

ステップ6: 複数冊の登録とグリッド表示の確認

1. さらに2〜3冊の書籍を登録します（ステータスを変えて登録すると、バッジの色分けも確認できます）。
2. 画面幅を変えて、グリッドレイアウトのレスポンス動作を確認します: - デスクトップ（1024px 以上）: 3列 - タブレット（768px〜1023px）: 2列 - スマホ（767px 以下）: 1列

ステップ7: Supabase ダッシュボードでのデータ確認

1. Supabase ダッシュボード (<https://supabase.com/dashboard>) にアクセスします。
2. Table Editor で **books** テーブルを開きます。
3. 登録した書籍のレコードが正しく保存されていることを確認します。

7. トラブルシューティング

問題1: データが表示されない（一覧が常に空）

症状: 書籍を登録したはずなのに、トップページに書籍が表示されない。「全 0 冊」のまま。

原因と対策:

1. Supabase の接続情報が正しくない

`.env.local` ファイルの内容を確認してください。

```
bash # .env.local NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbG...
```

- URL の末尾にスラッシュ `/` が余計についていないか確認してください。
- `.env.local` を変更した場合は、開発サーバー（`npm run dev`）を **再起動** する必要があります（Ctrl+C で停止し、再度 `npm run dev`）。

2. Row Level Security (RLS) の設定

Supabase のテーブルに RLS が有効になっていると、ポリシーが設定されていない場合はデータを取得できません。

Supabase ダッシュボードで確認: - Authentication → Policies → `books` テーブル - SELECT に対するポリシーが設定されているか確認 - テスト段階では一時的に RLS を無効にすることもできます（ダッシュボードの Table Editor → `books` テーブル → RLS を OFF に）

3. Supabase クライアントのインポートパスが間違っている

`app/page.tsx`（Server Component）では `@/lib/supabase/server` からインポートし、
`app/books/new/page.tsx`（Client Component）では `@/lib/supabase/client` からインポートする必要があります。逆にするとエラーになります。

4. ブラウザの開発者ツールで確認する

F12 → Console タブを開き、エラーメッセージがないか確認してください。また、Network タブで Supabase への API リクエストが成功しているか（ステータスコード 200）を確認してください。

問題2: フォーム送信でエラーが出る

症状:「登録する」ボタンを押すと、エラーメッセージ（「書籍の登録に失敗しました」）が表示される。

原因と対策:

1. テーブルのカラム名が一致していない

`insert` に渡すオブジェクトのキー名が、データベースのカラム名と一致していない可能性があります。ブラウザの Console にエラー詳細が表示されているはずなので確認してください。

```
// よくある間違い publishedDate: "2024-01-01" // NG: キャメルケース published_date: "2024-01-01" // OK: スネークケース (DB のカラム名に合わせる)
```

2. NOT NULL 制約違反

データベースの `title` や `author` カラムに NOT NULL 制約がかかっている場合、空文字列や `null` を送信するとエラーになります。BookForm のバリデーションで必須チェックを行っていますが、バリデーションをバイパスして送信した場合に発生します。

3. RLS ポリシーの INSERT が許可されていない

Supabase のテーブルに RLS が有効になっていて、INSERT のポリシーが設定されていない場合、データを挿入できません。SELECT と同様に、INSERT のポリシーも設定する必要があります。

4. Supabase のクライアント用ライブラリを使っているか確認

`app/books/new/page.tsx` は Client Component なので、`@/lib/supabase/client` を使います。
`@/lib/supabase/server` を使うと `cookies()` などのサーバー専用APIにアクセスしようとしてエラーになります。

問題3: 型エラーが出る

症状: TypeScript のコンパイルエラーが発生する。

原因と対策:

1. `Book` 型のインポート漏れ

`Cannot find name 'Book'.`

`Book` 型を使うファイルで、正しくインポートしているか確認してください。

```
tsx import { type Book } from "@/components/BookCard";
```

`type` キーワードを使っているのは「型だけをインポートする」という意味です。TypeScript 専用の構文で、コンパイル後の JavaScript からは削除されます。なくても動きますが、付けておくとインポートの意図が明確になります。

2. `BookStatus` 型のインポート漏れ

`Cannot find name 'BookStatus'.`

`StatusBadge.tsx` からエクスポートしている型なので:

```
tsx import { type BookStatus } from "@/components/StatusBadge";
```

3. Supabase の型定義との不一致

Supabase CLI で型を自動生成している場合、自動生成された型と手動で定義した `Book` 型に差異があるとエラーになることがあります。自動生成された型を使う方法は後の章で解説しますが、現段階では手動で定義した型で問題ありません。

4. `null` と `undefined` の混在

`Type 'undefined' is not assignable to type 'string | null'.`

Supabase から返るデータは `null` (`undefined` ではない) です。型定義で `string | undefined` ではなく `string | null` を使ってください。

5. `createClient` の名前衝突

サーバー用とクライアント用で同じ名前 `createClient` をインポートしていますが、ファイルが異なるので衝突しません。ただし、1つのファイルで両方使おうとすると衝突します。そのような場合はエイリアスを使います:

```
tsx import { createClient as createServerClient } from "@lib/supabase/server"; import {
createClient as createBrowserClient } from "@lib/supabase/client";
```

まとめ

この章では、以下の機能を実装しました。

- **一覧表示:** Server Component でSupabase からデータを取得し、BookCard を使ってグリッド表示
- **ステータス・評価の視覚化:** StatusBadge（色分けバッジ）、RatingStars（星評価）
- **ローディング状態:** loading.tsx とスケルトン UI
- **エラーハンドリング:** error.tsx とリトライ機能
- **新規登録:** BookForm でフォーム入力、バリデーション、Supabase への INSERT、リダイレクト

次の章では、**詳細表示（Read）**、**編集（Update）**、**削除（Delete）** を実装し、CRUD の残りの機能を完成させます。